pAIntry Backend API

Base URL: `https://<your-app>.onrender.com` (replace with the Render URL) **Interactive docs:** `<base>/docs` CORS is open, so the frontend can call it directly. All responses are JSON.

> The free Render instance sleeps after ~15 min idle. First request may take ~30 s.

GET /health

Checks the server and database. **Response:** `{"status": "ok"}`

GET /products

Seeded products from the database.

Query param	Type	Required	Notes
`category`	string	no	e.g. `yogurt`

Response: array of product objects.

GET /live-prices

Live prices from 3 Seattle QFC stores (Kroger API).

Query param	Type	Required	Default
`q`	string	yes	
`limit`	int	no	5

Example: `GET /live-prices?q=greek%20yogurt&limit=3`

Response: array of

```
{
  "name": "...", "brand": "...", "category": "...", "upc": "...", "size": "32 oz",
  "prices": {"<storeId>": 5.99},
  "cheapest_store": "...", "price": 5.99,
  "protein_g_per_package": 80, "cost_per_protein_g": 0.07,
  "allergens": ["milk"], "image": "https://...", "source": "kroger"
}
```

Errors: `503` if Kroger is unavailable.

GET /users/{user_id}

One user profile (allergies, goals, budget). **Errors:** `400` invalid id, `404` not found.

GET /users/{user_id}/pantry

The user's pantry items, sorted by `expires_at` (soonest first).

POST /users/{user_id}/pantry

Add a pantry item. Returns the created item (201).

```
{
  "name": "Milk",
  "quantity": 1,
  "unit": "item",
  "product_id": null,
  "expires_at": "2026-10-20T00:00:00Z"
}
```

Only `name` is required.

GET /users/{user_id}/savings

```
{"user_id": "...", "total_saved": 12.34, "entries": 3}
```

POST /convince-me (main feature)

Decides whether to buy a product or swap to a cheaper or healthier one, and explains why with AI (Gemini).

Body (`user_id` plus one of `query` / `product_id` / `barcode`): | Field | Type | Default | Notes | |---|---|---|---| | `user_id` | string | required | | | `query` | string | null | free text, e.g. `"greek gods yogurt"` | | `product_id` | string | null | DB product id | | `barcode` | string | null | | | `live` | bool | true | use live QFC prices for `query` | | `log` | bool | true | `false` = don't save to history/savings (use for demos) |

Can also pass `?log=false` in the URL.

Example:

```
{"user_id": "<id>", "query": "greek gods yogurt", "log": false}
```

Key response fields: | Field | Meaning | |---|---| | `verdict` | e.g. `"Swap"` or keep/buy | | `savings` | dollars saved by swapping | | `better_alternative` | the suggested product (object with `name`, etc.) | | `reasons` | AI-written explanation | | `source` | `"gemini"` or `"engine"` (fallback) | | `data_sources` | where prices/nutrition came from | | `engine` | raw calculated facts, incl. `price_source`, `sample_data`, `expiring_soon` | | `logged` | whether it was saved |

Errors: `400` missing product info or bad id, `404` user/product not found.

Frontend fetch example

```
const API = "https://<your-app>.onrender.com";
const res = await fetch(`${API}/convince-me`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ user_id: USER_ID, query: "greek gods yogurt", log: false }),
```

```
});
const data = await res.json();
```